

Automatische Auswertung von Leistungserhebungen zur unmittelbaren Diagnostik im (Informatik-)Unterricht

Kolloquium Institut für Psychologie Jena

Mike Barkmin & Matthias Kramer ■ 15. Januar 2020

Inhaltsverzeichnis

1. Einleitung
2. Motivation
3. Vorbetrachtungen
4. Das Online-Assessment-Tool
5. Zusammenfassung
6. Ausblick



Abbildung: Bild von TeroVesalainen unter Pixabay License via Pixabay


Einleitung

Wer sind wir?



Matthias Kramer



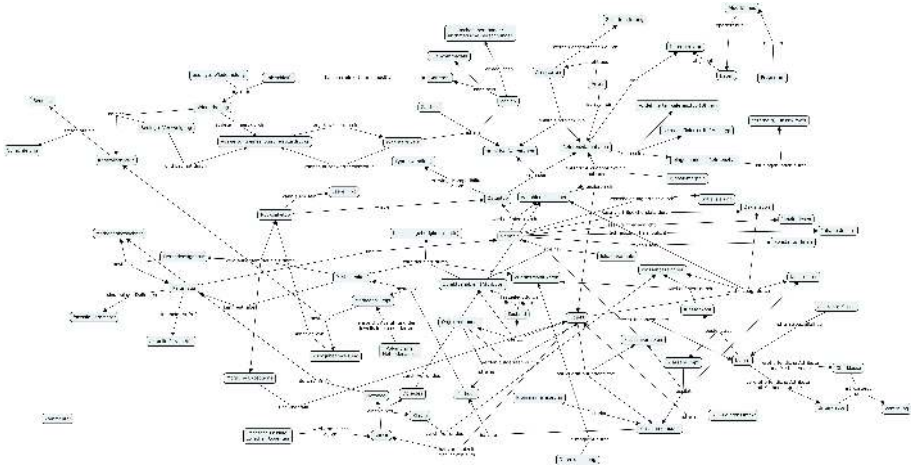
Lehrstuhl für Didaktik der Informatik

Universität Duisburg-Essen



Mike Barkmin

Woran forschen wir?




Motivation

Motivation



Abbildung: Bild von Ag Ku unter Pixabay License via Pixabay

Motivation

- Bei Erhebungen haben wir Probleme festgestellt, die so auch schon in der Literatur beschrieben wurden



Abbildung: Bild von Ag Ku unter Pixabay License via Pixabay

Motivation

- Bei Erhebungen haben wir Probleme festgestellt, die so auch schon in der Literatur beschrieben wurden
- Zeitaufwand bei der Digitalisierung und späterer Auswertung



Abbildung: Bild von Ag Ku unter Pixabay License via Pixabay

Motivation

- Bei Erhebungen haben wir Probleme festgestellt, die so auch schon in der Literatur beschrieben wurden
- Zeitaufwand bei der Digitalisierung und späterer Auswertung
- Größere Stichproben sehr aufwändig zu bewerkstelligen



Abbildung: Bild von Ag Ku unter Pixabay License via Pixabay

Motivation

- Bei Erhebungen haben wir Probleme festgestellt, die so auch schon in der Literatur beschrieben wurden
- Zeitaufwand bei der Digitalisierung und späterer Auswertung
- Größere Stichproben sehr aufwändig zu bewerkstelligen
- Komplexere aber realistischere Aufgabentypen schwer zu realisieren (Stichwort: Programmieren auf Papier)



Abbildung: Bild von Ag Ku unter Pixabay License via Pixabay


Vorbetrachtungen

Vorbetrachtungen I

- Webapplikation (WA)
- Vorerst keine Notwendigkeit für Nutzeraccount ⇒ Zugang zum Test nur durch Besuch einer Webseite & Token (NUA)
- Analyse der Problemlösefähigkeiten durch Untersuchung der Nutzerinteraktion mit den Aufgaben ⇒ Tracken der Nutzerinteraktion mit den Items (Anzahl der Versuche, Zeit, Reihenfolge, Lösungsansätze etc.) (UIT)
- Datenschutz: entweder verschlüsselte Ablage der Daten oder mind. Server an deutschen Hochschulen (DS)
- Möglichkeiten zur selbstständigen Item- und Testerstellung (ITE)
- Evtl. Erweiterbarkeit um neue Item-Formate ⇒ direkte Ansprachemöglichkeit (CE)
- Möglichkeit zum Herunterladen der Daten in bspw. CSV-Format zur Weiterverarbeitung durch vorgefertigte R-Skripte oder bereits serverseitige Verarbeitung

Vorbetrachtungen II

	WA	NUA	UIT	DS	ITE	CE
JACK (Goedicke und Striewe, 2017)	(✓)	✗	✗	✓	✓	✓
VILLE (Rajala u. a., 2016)	✓	✗	?	✓	✓	✓
TRAKLA2 (Laakso u. a., 2004)	(✓)	✗	✓	✓	?	?
BOSS2 (Joy u. a., 2005)	(✓)	✗	✗	✓	?	?
ProGoSS (Gluga u. a., 2011)	(✓)	✗	✗	✗	?	?
QuizJET (Hsiao u. a., 2008)	✓	✓	✗	✗	?	?

- Zusätzlich wurden weitere Systeme (Mooshak, Bottlenose, CourseMarker, WeBWorK uvm.) betrachtet, keines für unsere Zwecke schnell einsetzbar
- $\Rightarrow$ Eigene Systementwicklung war notwendig


Das Online-Assessment-Tool

Das Online-Assessment-Tool

1. Einleitung

2. Motivation

3. Vorbetrachtungen

4. Das Online-Assessment-Tool

   4.1 Konzeptionelle Realisierung

   4.2 Technische Realisierung

   4.3 Item-Ebene

   4.4 Test-Ebene

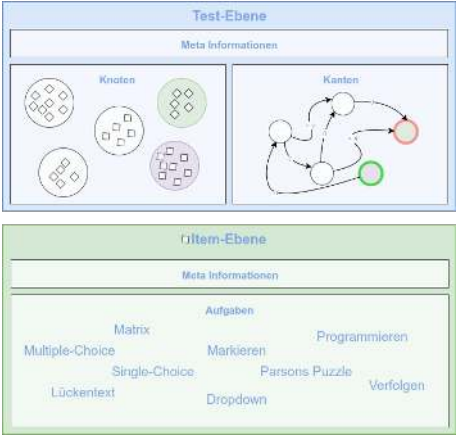
5. Zusammenfassung

6. Ausblick



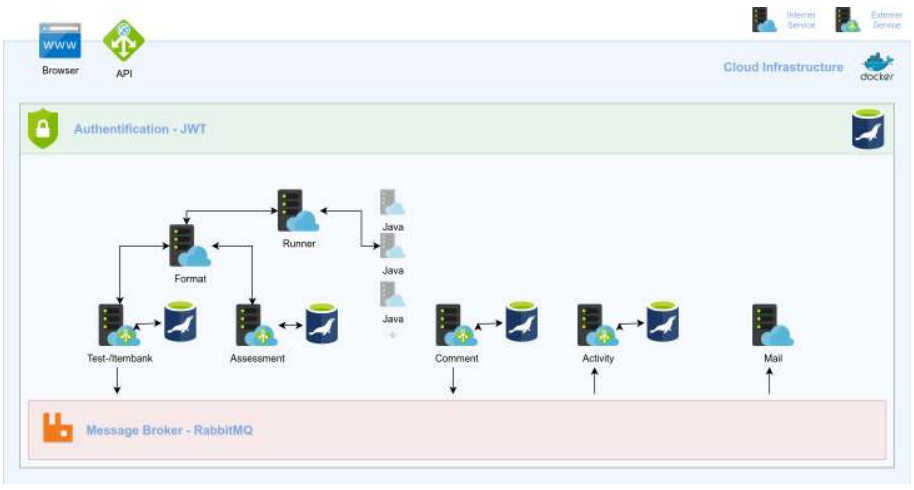
Abbildung: Bild von TeroVesalainen unter Pixabay License via Pixabay

Konzeptionelle Realisierung



{mike.barkmin, matthias.kramer}@uni-due.de

Technische Realisierung



{mike.barkmin, matthias.kramer}@uni-due.de

Item-Ebene

4. Das Online-Assessment-Tool

4.1 Konzeptionelle Realisierung

4.2 Technische Realisierung

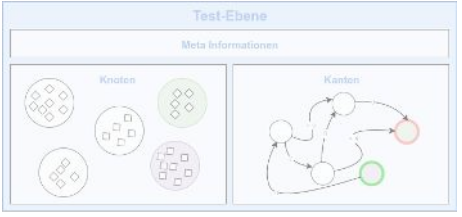
4.3 Item-Ebene

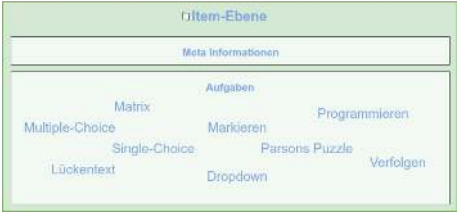
4.3.1 Analog zu Digital

4.3.2 Komplexere Authentische Aufgabentypen

4.3.3 Betrachtung des Lösungswegs

4.4 Test-Ebene





{mike.barkmin, matthias.kramer}@uni-due.de

Analog zu Digital I

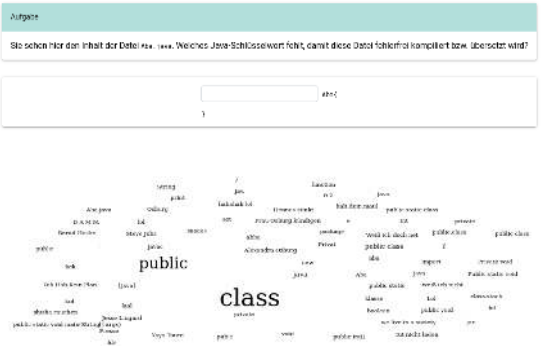
- Single/Multiple-Choice
  - Analoge Aufgabentypen digitalisieren
  - Dadurch schnellere Auswertung
  - Direkte Visualisierung
  - (Versehentliches Vergessen einer Antwort kann abgefangen werden)



{mike.barkmin, matthias.kramer}@uni-due.de

Analog zu Digital II

- Lückentext
  - „Handschrifterkennung“ entfällt
  - Auswertung kann direkt durch reguläre Ausdrücke realisiert werden
  - Diagnostische Visualisierung als Wortwolke
  - Siehe Striewe u. a. (2017)



{mike.barkmin, matthias.kramer}@uni-due.de

Komplexere Authentische Aufgabentypen

- Wünschenswert auch authentische Aufgabentypen einsetzen zu können
- Dazu haben wir einen Runner implementiert
- Quelltext wird bei uns auf dem Server kompiliert und gegen Unittests getestet

```java
import java.util.Scanner;

public class HelloWorld {

      public static void main(String[] args) {

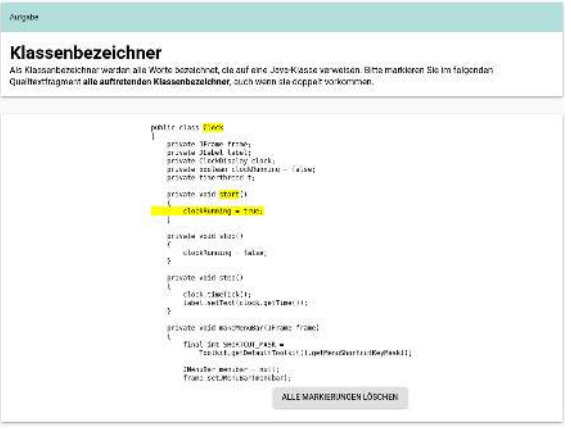
            // Creates a reader instance which takes
            // input from standard input - keyboard
            Scanner reader = new Scanner(System.in);
            System.out.print("Enter a number: ");

            // nextInt() reads the next integer from the keyboard
            int number = reader.nextInt();

            // println() prints the following line to the output scre
            System.out.println("You entered: " + number);
      }
}
```

Entwicklung Komplexerer Aufgabentypen: Highlighting I

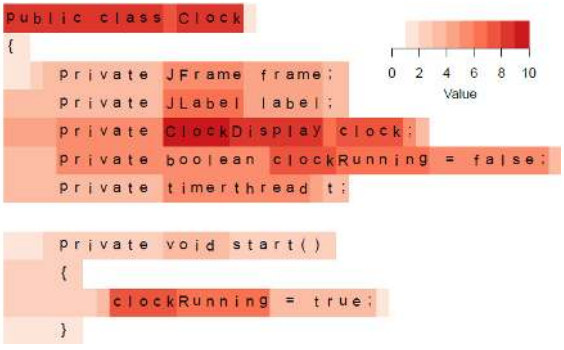
- Vergleichsweise einfacher Itemtyp, trotzdem realistisch
- Verbindet konzeptuelles Wissen mit Repräsentation in formalen Sprachen
- Idee: Markiere alle Vorkommen von `<Konzept>` im gegebenen Quelltext
- Notwendige Aktionen: Verändern des CSS-Attributs (Hintergrundfarbe), Speichern der Indizes



{mike.barkmin, matthias.kramer}@uni-due.de

Entwicklung Komplexerer Aufgabentypen: Highlighting II

- Zusätzlich Bestimmung eines Bewertungsmaßes
- Diagnostische Visualisierung für Gruppengröße: Heatmap
- Beschrieben in Kramer u. a. (2018)



Betrachtung des Lösungswegs

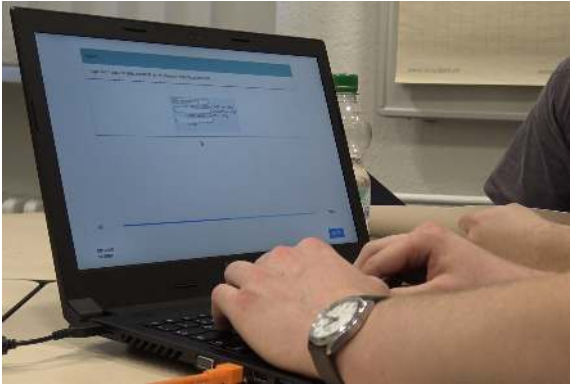
- Durch die Betrachtung des Endergebnisses gehen wichtige Information verloren
- Der Lösungsweg könnte zusätzlich betrachtet werden



Abbildung: Bild von Bhuvanesh S unter Pixabay License via Pixabay

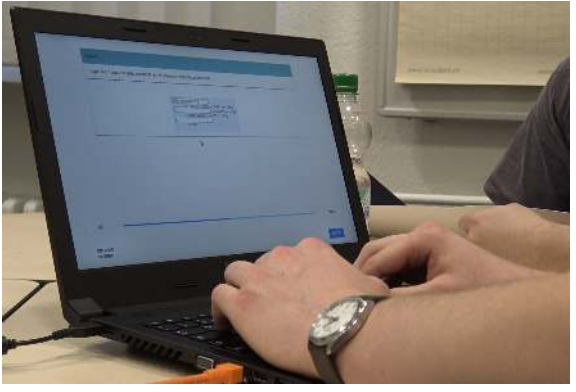
Betrachtung des Lösungswegs

- Durch die Betrachtung des Endergebnisses gehen wichtige Information verloren
- Der Lösungsweg könnte zusätzlich betrachtet werden
- Lösung: Videoaufzeichnung in HD der Bearbeitung
  - 8 Probanden ca. 30 min ~ 140GB



Betrachtung des Lösungswegs

- Durch die Betrachtung des Endergebnisses gehen wichtige Information verloren
- Der Lösungsweg könnte zusätzlich betrachtet werden
- Lösung: Videoaufzeichnung in HD der Bearbeitung
  - 8 Probanden ca. 30 min ~ 140GB
- Alternative Lösung: Aufzeichnung der Interaktionen mit dem Assessment-Tool
  - ca. 500 Probanden ~ 20MB



Aufzeichnen des Lösungswegs

- Action: Ist eine Aktion die vom Benutzer ausgeführt wird
- Reducer: Entscheidet wie sich der Store abhängig von der Aktion ändern soll
- Store: Eine Sammlung von Daten
- UI: Wird abhängig vom Store dargestellt

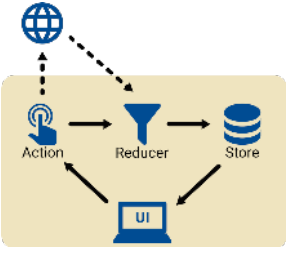
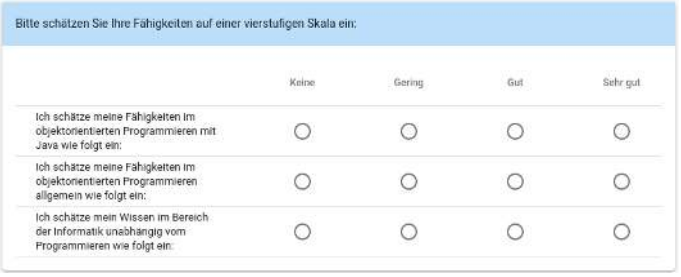


Abbildung: Action-Reducer-Store angelehnt an https://redux.js.org

Aufzeichnung des Lösungswegs - Beispiel

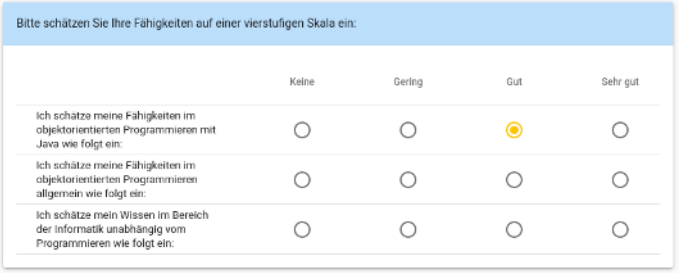
- Jeder Aufgabentyp funktioniert nach dem gleichen Prinzip
- Jeder Aufgabentyp hat eigens definierte Actions
- Actions des Matrix-Single-Choice Aufgabentyps
  - CHECK_CHOICE



Matrix-Single-Choice Aufgabentyp

Aufzeichnung des Lösungswegs - Beispiel

Action	Data	Time (in ms)
CHECK_CHOICE	[0, 2]	1100



Matrix-Single-Choice Aufgabentyp

Aufzeichnung des Lösungswegs - Beispiel

Action	Data	Time (in ms)
CHECK_CHOICE	[0, 2]	1100
CHECK_CHOICE	[1, 2]	2000



Matrix-Single-Choice Aufgabentyp

Nutzung des Lösungswegs für Visualisierungen

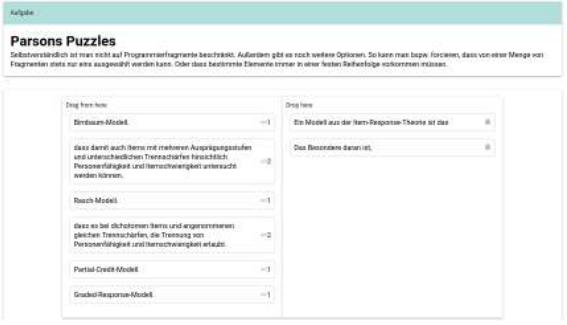
Nutzung des Lösungswegs für Visualisierungen - Parsons Puzzle



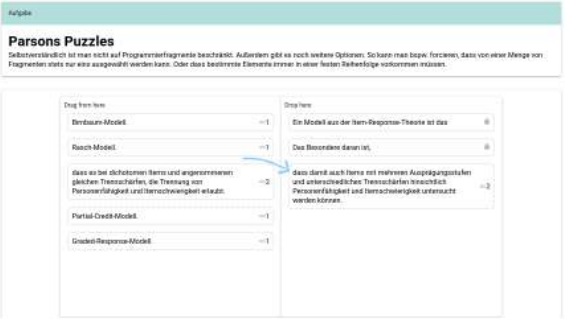
Initialer Zustand

- Basiert auf der Arbeit von Parsons und Haden (2006)
- Actions für Parsons Puzzles
  - MOVE_FROM_SOURCE_TO_USER (sourceId, userId)
  - MOVE_FROM_USER_TO_SOURCE (userId, sourceId)
  - MOVE_WITHIN_USER (userId1, userId2)
- Was passiert wenn die Action MOVE_FROM_SOURCE_TO_USER (0, 1) ausgeführt wird?

Nutzung des Lösungswegs für Visualisierungen - Parsons Puzzle



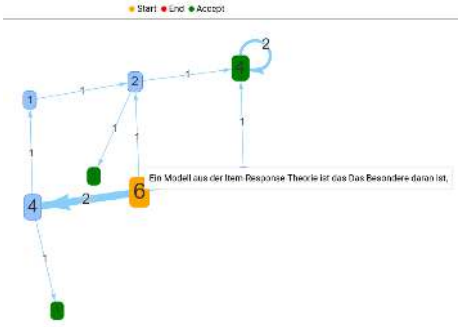
Initialer Zustand



Nach Ausführung der Action MOVE_FROM_SOURE_TO_USER (0, 1)

Nutzung des Lösungswegs für Visualisierungen - Parsons Puzzle

- Die Ausführung einer Action führt zu einer Zustandsänderung der Daten
- Jeder Zustand des Parsons-Puzzle ist durch einen Knoten symbolisiert
- Jeder Kante ist die Ausführung einer Action
- Die Zahl/Dicke der Kanten/Knoten symbolisiert wie häufig sie erreicht wurden



Visualisierung von 6 Lösungswegen angelehnt an Helminen u. a. (2012)

Nutzung des Lösungswegs zum Aufdecken kognitiver Strukturen - Quelltext memorieren (Barkmin u. a., 2017)




```
class Datei {
    String name;
    String pfad;

    Datei(String name, String pfad) {
        this.name = name;
        this.pfad = pfad;
    }

    void umbenennen(String name) {
        this.name = name;
    }
}
```


{mike.barkmin, matthias.kramer}@uni-due.de

Nutzung des Lösungswegs zum Aufdecken kognitiver Strukturen - Quelltext memorieren



- Actions für Quelltext memorieren
  - INSERT_CHAR (charId, pos)
  - REMOVE_CHAR (pos)
  - OPEN_SOURCE_CODE
  - CLOSE_SOURCE_CODE

Nutzung des Lösungswegs zum Aufdecken kognitiver Strukturen - Quelltext memorieren

- Viele Aktionen können unübersichtlich werden
- Es wird jeder Tastendruck aufgezeichnet
- Daher müssen Aktionen zusammengefasst werden
- Hier in Merkphasen (Blau), Schreibphasen (Grün) und Pausephasen (Hellblau)

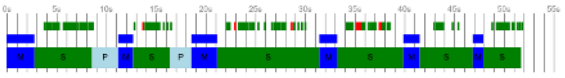


Abbildung: Zeitleiste einer Bearbeitung

	Phase	Zeit in s	LS	G	Inhalt
5	M	2.50			
6	S	5.71	33	11	␣␣␣␣public␣class␣Haus␣{ ␣␣␣␣private␣int␣nummer; ␣␣␣␣private␣String␣
7	P	2.43			
8	S	3.01	-6	1	␣␣␣␣public␣class␣Haus␣{ ␣␣␣␣private␣int␣nummer; ␣␣␣␣private␣String␣Strasse;
9	P	2.43			
		8.72 (4.86)	-39	1	
10	M	2.15			
11	P	2.19			
12	S	5.87	0	8	␣␣␣␣public␣class␣Haus␣{ ␣␣␣␣private␣int␣nummer; ␣␣␣␣private␣String␣Strasse;

Abbildung: Transkript einer Bearbeitung

{mike.barkmin, matthias.kramer}@uni-due.de

Auch für psychologische Tests nutzbar

- Digitale Version der Corsi-Blöcke (Corsi, 1972)
- Separat entwickelt (Offline-Anwendung für Mac, Windows und Linux)
- Blöcke können beliebig angeordnet werden
- Basiert auch auf Action-Reducer-Store
- Actions
  - MOUSE_MOVED (x, y)
  - CORNER_PRESSED
  - CORNER_RELEASED
  - BLOCK_PRESSED (id)
  - BLOCK_RELEASED (id)

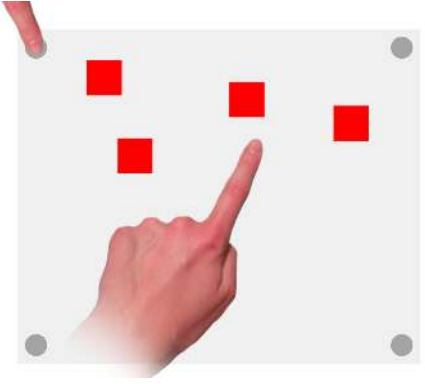


Abbildung: Digitale Version der Corsi-Blöcke

{mike.barkmin, matthias.kramer}@uni-due.de

Auch für psychologische Tests nutzbar

- Digitale Version der Corsi-Blöcke (Corsi, 1972)
- Separat entwickelt (Offline-Anwendung für Mac, Windows und Linux)
- Blöcke können beliebig angeordnet werden
- Basiert auch auf Action-Reducer-Store
- Actions
  - MOUSE_MOVED (x, y)
  - CORNER_PRESSED
  - CORNER_RELEASED
  - BLOCK_PRESSED (id)
  - BLOCK_RELEASED (id)

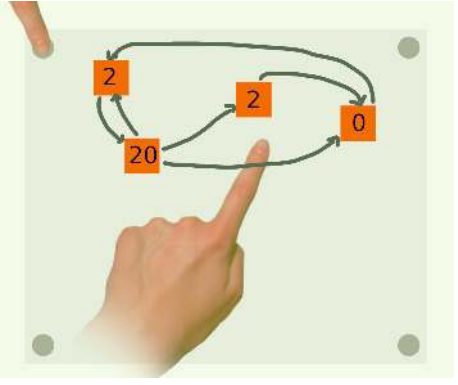


Abbildung: Digitale Version der Corsi-Blöcke

Test-Ebene

4. Das Online-Assessment-Tool

4.1 Konzeptionelle Realisierung

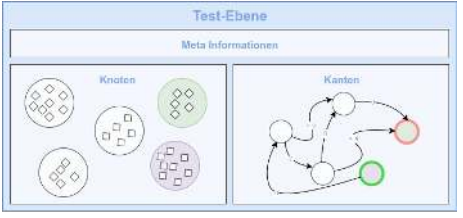
4.2 Technische Realisierung

4.3 Item-Ebene

4.4 Test-Ebene

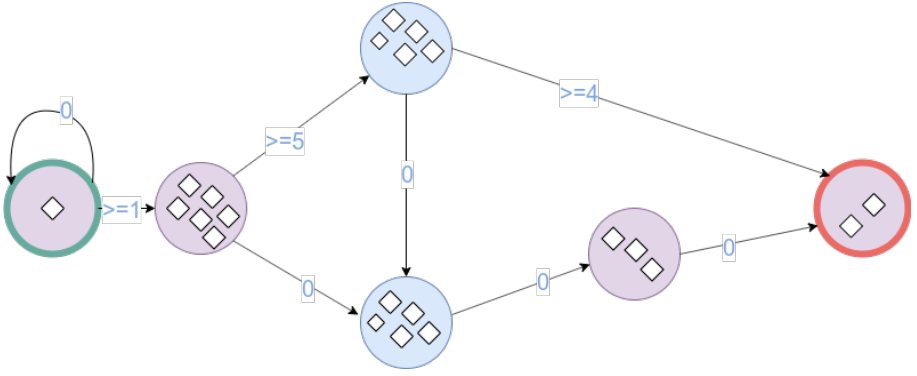
4.4.1 Graph eines Tests

4.4.2 Knoten-Typen



{mike.barkmin, matthias.kramer}@uni-due.de

Graph eines Tests



Verschiedene Knoten-Typen

Linear



⚙

✓ Muss korrekt sein

✓ Zufällig

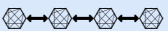
🏁

✓ Zeit

✓ Items

✓ Punkte

Sprung



⚙

✓ Muss korrekt sein

✓ Zufällig

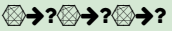
🏁

✓ Zeit

✓ Items

✓ Punkte

Adaptiv



🏁

✓ Punkte

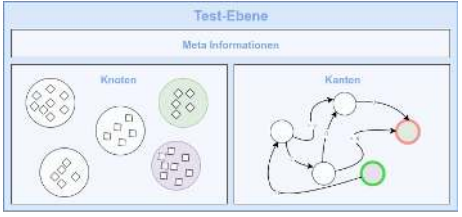
✓ Präzision

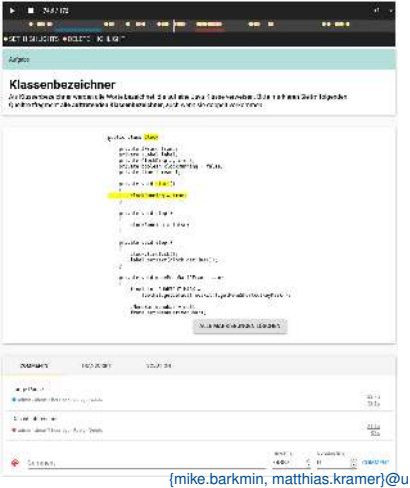
🏁 Abbruchkriterium, ⚙ Einstellung

{mike.barkmin, matthias.kramer}@uni-due.de

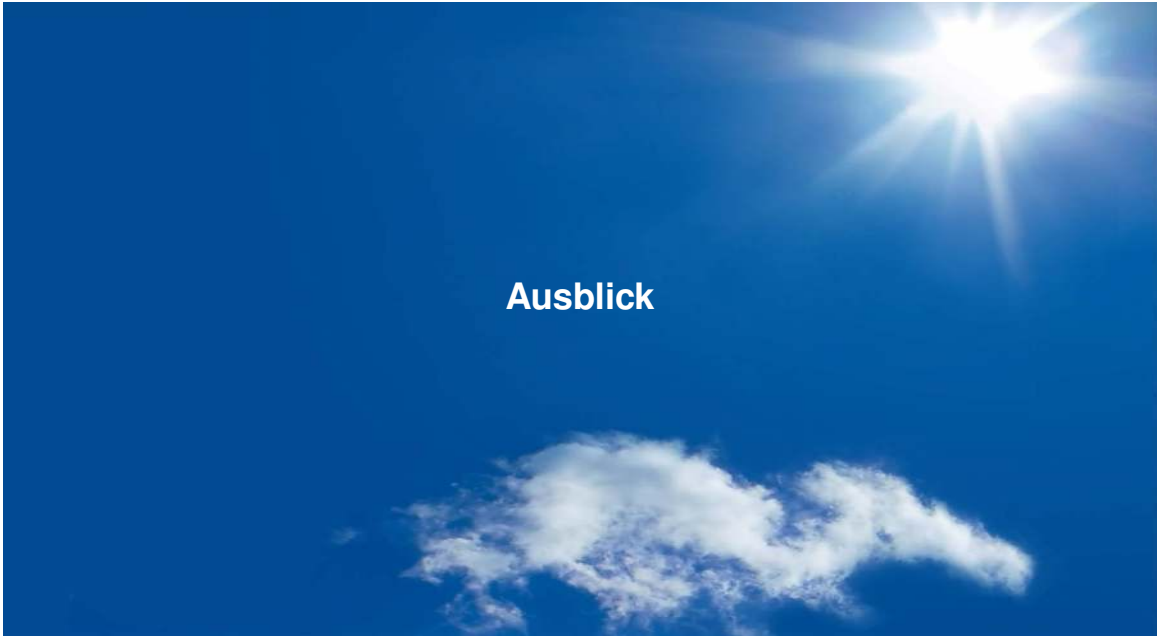

Zusammenfassung

Zusammenfassung





{mike.barkmin, matthias.kramer}@uni-due.de


Ausblick

Weitere Entwicklungen

Verbesserung des Datenschutzes

Verschlüsselter Knoten-Typen

Assessment herauslösen



Abbildung: Bild von Pete Linforth unter Pixabay License via Pixabay

Adaptivität

Neuer Knoten-Typ

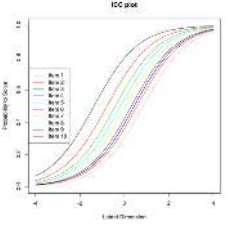


Abbildung: Bild von Produnis unter Public Domain via Wikimedia

Mustererkennung

Automatische Auswertung des Bearbeitungsprozesses

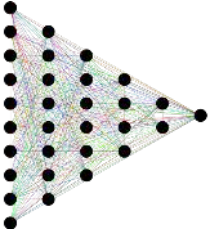
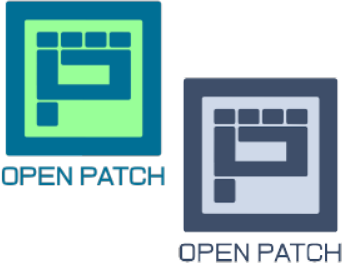


Abbildung: Bild von GDJ unter Pixabay License via Pixabay

{mike.barkmin, matthias.kramer}@uni-due.de

Falls Sie das Werkzeug einsetzen/mitentwickeln möchten

- Sie sind zur Mitarbeit eingeladen!
- 🦊: Öffentliches Gitlab-Repository
  https://gitlab.com/openpatch
- Verwendete Technologien
  - Backend: Python, Flask, MariaDB
  - Frontend: JavaScript, React





Vielen Dank für die Aufmerksamkeit!

Kontakt
Mike Barkmin & Matthias Kramer
Didaktik der Informatik
Universität Duisburg-Essen
Schützenbahn 70, 45127 Essen
{mike.barkmin, matthias.kramer}@uni-due.de
http://udue.de/{mba, mk}

Literaturverzeichnis I

Barkmin, Mike u. a. (2017). „Code Structure Difficulty in OOP: An Exploration Study Regarding Basic Cognitive Processes“. In: *Proceedings of the 17th Koli Calling Conference on Computing Education Research*. Koli Calling '17. New York, NY, USA: ACM, S. 185–186. ISBN: 978-1-4503-5301-4.

Corsi, P. M. (1972). *Human Memory and the Medial Temporal Region of the Brain*. McGill Theses.

Gluga, Richard u. a. (12. Dez. 2011). „An Architecture for Systematic Tracking of Skill and Competence Level Progression in Computer Science“. In: *2nd Annual International Conferences on Computer Science Education: Innovation and Technology (CSEIT 2011)*. Annual International Conferences on Computer Science Education: Innovation and Technology. Global Science & Technology Forum (GSTF).

Goedicke, Michael und Michael Striewe (2017). „10 Jahre automatische Bewertung von Programmieraufgaben mit JACK - Rückblick und Ausblick“. In: *Lecture Notes in Informatics*. INFORMATIK 2017. Gesellschaft für Informatik, Bonn. ISBN: 978-3-88579-669-5.

Literaturverzeichnis II

Helminen, Juha u. a. (2012). „How Do Students Solve Parsons Programming Problems?: An Analysis of Interaction Traces“. In: *Proceedings of the Ninth Annual International Conference on International Computing Education Research*. ICER '12. New York, NY, USA: ACM, S. 119–126. ISBN: 978-1-4503-1604-0.

Hsiao, I-Han u. a. (1. Jan. 2008). „Web-Based Parameterized Questions for Object-Oriented Programming“. In:

Joy, Mike u. a. (Sep. 2005). „The Boss Online Submission and Assessment System“. In: *Journal on Educational Resources in Computing* 5.3.

Kramer, Matthias u. a. (2018). „Automatic Assessment of Source Code Highlighting Tasks: Investigation of Different Means of Measurement“. In: *Proceedings of the 18th Koli Calling International Conference on Computing Education Research*. Koli Calling '18. 00000. New York, NY, USA: ACM, 8:1–8:10. ISBN: 978-1-4503-6535-2.

Laakso, Mikko u. a. (2004). „Automatic Assessment of Exercises for Algorithms and Data Structures–a Case Study with TRAKLA2“. In: *Proceedings of the 4th Finnish/Baltic Sea Conference on Computer Science Education*, S. 28–36.

Literaturverzeichnis III

Parsons, Dale und Patricia Haden (2006). „Parson's Programming Puzzles: A Fun and Effective Learning Tool for First Programming Courses“. In: *Proceedings of the 8th Australasian Conference on Computing Education - Volume 52*. ACE '06. Darlinghurst, Australia, Australia: Australian Computer Society, Inc., S. 157–163. ISBN: 978-1-920682-34-7.

Rajala, Teemu u. a. (2016). „Automatically Assessed Electronic Exams in Programming Courses“. In: *Proceedings of the Australasian Computer Science Week Multiconference*. ACSW '16. New York, NY, USA: Association for Computing Machinery. ISBN: 978-1-4503-4042-7.

Striewe, Michael u. a. (2017). „Ein Lückentext-Test Zur Beherrschung Einer Programmiersprache“. In: *Bildungsräume 2017*. Hrsg. von Christoph Igel u. a. Gesellschaft für Informatik, Bonn, S. 261–266.